



Kimlik Doğrulama, Yetkilendirme ve RBAC

Kubernetes ve Konteyner Dünyası



Bu Konuda

- API istek zinciri: kimlik doğrulama, yetkilendirme, kabul denetimi
- Kullanıcı ve ServiceAccount kimlikleri
- RBAC kaynakları ve en az ayrıcalık
- Yetki yükseltme riskleri: `escalate`, `bind`, `impersonate`, Secret ve CSR
- ServiceAccount token ve CSR akışları
- Yetkilendirme modları ve namespace katmanları

API İstek Zinciri





Kimlik doğrulama, yetkilendirme ve kabul denetimi

Aşama	Yanıtladığı soru	Ret örneği
Kimlik doğrulama ("authentication")	İstek hangi kullanıcı, grup veya ServiceAccount'tan geliyor?	Kimlik doğrulanamazsa 401 Unauthorized
Yetkilendirme ("authorization")	Bu kimlik, bu fiili bu hedefte yapabilir mi?	Hiçbir politika izin vermezse 403 Forbidden
Kabul denetimi ("admission")	İzinli yazma isteğinin nesne içeriği politikaya uygun mu?	Nesne değiştirilebilir ya da reddedilebilir

Kabul denetimi yetkilendirmeden sonra çalışır; yalnızca okuma yapan isteklerde çalışmaz. İzin verilen ve kabul denetiminden geçen nesne doğrulanıp kalıcı depoya yazılır. [\[1,3,4\]](#)



Kullanıcı ve ServiceAccount kimlikleri

Özellik	Kullanıcı / grup	ServiceAccount
Temsil ettiği	İnsan, ekip veya dış otomasyon	Workload ya da makine kimliği
Kayıt yeri	Harici kimlik sistemi; Kubernetes'te <code>User</code> nesnesi yok	Namespace kapsamlı API nesnesi
Kimlik bilgisi	OIDC token'ı, istemci sertifikası vb.	Genellikle kısa ömürlü JWT token
RBAC konusu	<code>User</code> veya <code>Group</code> subject'i	<code>ServiceAccount</code> subject'i
Pod varsayılanı	Uygulanmaz	Ad verilmezse namespace'in <code>default</code> hesabı



Yetkilendirme kararı

Yetkilendirme kararı yalnız "kim?" sorusuna bakmaz; şu istek niteliklerini birlikte değerlendirir: ^[3]

- kullanıcı, gruplar ve kimlik doğrulayıcının ek alanları;
- API grubu, resource ve varsa alt kaynak (`pods/log` , `deployments/scale`);
- `get` , `list` , `watch` , `create` , `patch` , `delete` gibi Kubernetes fiili;
- namespace ve tek nesne isteklerinde nesne adı;
- API resource'u olmayan hedeflerde URL yolu ve HTTP fiili

`get` , `list` ve `watch` dönen veri açısından eşdeğer hassasiyet taşıyabilir: `list secrets` yanıtı Secret verilerini de içerir. ^[3]

Kimlik doğrulanmış olmak erişim hakkı vermez; isteğin ilerlemesi için bir yetkilendiricinin açıkça izin vermesi gerekir.

RBAC





Dört RBAC kaynağı

Kaynak	Kapsam	İşi
Role	Tek namespace	Resource + fiil izinlerini tanımlar
ClusterRole	Cluster nesnesi	Cluster kapsamlı, kaynak bazlı olmayan veya yeniden kullanılabilir namespaced izin tanımlar
RoleBinding	Tek namespace	Role'ü ya da ClusterRole'ü yalnız binding namespace'inde subject'e verir
ClusterRoleBinding	Cluster geneli	ClusterRole izinlerini cluster çapında verir

Role / **ClusterRole** **ne yapılabileceğini**, binding ise bağlanan rolü **kimin nerede** yapabileceğini belirler. RBAC kuralları yalnız eklemelidir; bir "deny" kuralı yoktur. ^[5]

Bir ClusterRole + birçok namespace'te bu role RoleBinding oluşturmak her namespace'te ayrı Role tanımlamasının önüne geçer ve kaynak merkezileştirilmesi sağlar. ^[5]



Role ve ClusterRole nesne yapısı

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-okuyucu
  namespace: ekip-a
rules:
- apiGroups:
  - ""
  resources:
  - "pods"
  verbs:
  - "get"
  - "list"
  - "watch"
```

- `rules[]` : her öge hedef API grubu, resource ve fiilleri birlikte tanımlar
- `apiGroups: [""]` : core API grubu; `resources` : API resource adları; `verbs` : izin verilen Kubernetes fiilleri
- `ClusterRole` : `kind` değişir ve `metadata.namespace` kalkar; `rules` biçimi aynı kalır

Role ve ClusterRole izin tanıımıdır; bir binding oluşturulmadıkça hiçbir kimlik bu izinleri kazanmaz.^[5]



RoleBinding ve ClusterRoleBinding nesne yapısı

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: raporlayici-pod-okuma
  namespace: ekip-a
subjects:
- kind: ServiceAccount
  name: raporlayici
  namespace: ekip-a
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-okuyucu
```

- `subjects []` : bir veya daha çok `User` , `Group` ya da `ServiceAccount`
- `roleRef` : bağlanan tek `Role` / `ClusterRole` ; **oluşturulduktan sonra değiştirilemez**
- `RoleBinding` : etkili kapsam `metadata.namespace` ; aynı namespace'teki Role'ü veya bir ClusterRole'ü bağlayabilir
- `ClusterRoleBinding` : namespace içermez, yalnız ClusterRole bağlar ve izni cluster çapında verir^[5]



En az ayrıcalık (least privilege)

Tasarım kararı	Dar seçim	Sakıncalı geniş seçim
Yer	Gereken namespace'te RoleBinding	Gereksiz ClusterRoleBinding
Kimlik	Uygulamaya özel ServiceAccount	Namespace'in ortak <code>default</code> hesabı
Eylem	Açık <code>apiGroups</code> , <code>resources</code> , <code>verbs</code>	* jokeri ve gelecekte eklenecek resource'lar
Nesne	Uygunsa <code>resourceNames</code> ile belirli ad	Aynı türdeki her nesne
İşletim	Düşük yetkili günlük hesap	Sürekli <code>cluster-admin</code> / <code>system:masters</code>

- `resourceNames`, üst düzey `create` ve `deletecollection` isteklerini adla kısıtlayamaz; `list` / `watch` için istemcinin eşleşen `metadata.name` field selector göndermesi gerekir.^[5]
- Önce iş akışının gerektirdiği API çağrıları çıkarılır; rol yalnız bu çağrıları kapsayacak şekilde yazılır ve düzenli olarak yeniden incelenir^[6]



Yetki yükseltme ve veri riskleri

İzin	Yetki yükseltme veya veri riski
Role/ClusterRole'da <code>escalate</code>	Kişinin kendisinde olmayan izinleri role yazabilmesi
Role/ClusterRole'da <code>bind</code>	Kişinin kendisinde olmayan izinleri başkasına bağlayabilmesi
<code>impersonate</code>	Başka kullanıcı, grup veya ServiceAccount haklarıyla işlem yapabilme
Secret'larda <code>list</code> / <code>watch</code>	Dönen nesnelerdeki Secret içeriğini açığa çıkarma
Workload oluşturma	Namespace'teki Secret'ları bağlama ve ServiceAccount haklarını kullanma
CSR oluşturma + istemci CSR'ı onaylama	API'ye kabul edilen yeni istemci kimliği çıkarabilme
<code>serviceaccounts/token</code> 'da <code>create</code>	Var olan ServiceAccount için TokenRequest oluşturabilme

RBAC, rol yazma ve bağlamada varsayılan olarak kendi yetkisinin ötesine geçmeyi engeller; `escalate` ve `bind` bu korumayı aşar.^[5] Secret, workload, impersonation, token ve CSR izinleri bu yüzden salt "yönetim" yetkisi gibi değerlendirilmez.^[3,6]



escalate : rol yazma korumasını aşma

escalate ayrı bir API nesnesi değil, Role / ClusterRole içindeki özel bir yetkilendirme fiilidir.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: rol-duzenleyici
  namespace: ekip-a
rules:
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["roles"]
  verbs: ["create", "update", "patch", "escalate"]
```

- create / update / patch , Role nesnesini oluşturur ya da düzenler
- escalate , kullanıcının kendisinde olmayan izinleri de oluşturulan/düzenlenen Role'e ekleyebilmesini sağlar
- ClusterRole oluşturulması/düzenlenmesi için cluster kapsamlı izin ve resources: ["clusterroles"] gerekir

Örnek etki: Secret okuma izni olmayan bir rol oluşturucusu/düzenleyicisi, bu izni yeni bir Role'e ekleyebilir. ^[5]



bind : rol bağlama korumasını aşma

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: view-baglayici
rules:
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["rolebindings"]
  verbs: ["create"]
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["clusterroles"]
  verbs: ["bind"]
  resourceNames: ["view"]
```

- İlk kural RoleBinding oluşturur; ikinci kural, kullanıcı bu rolün izinlerine sahip olmasa da yalnız `view` ClusterRole'ünü bağlamaya izin verir
- `bind` tek başına binding oluşturamaz; `create` / `update` gibi binding yazma izni de gerekir
- `resourceNames` kaldırılırsa herhangi bir ClusterRole bağlanabilir

Bu ClusterRole `ekip-a` namespace'inde RoleBinding ile verildiğinde kullanıcı, kendisinde `view` izinleri olmasa bile başkasına `view` rolünü verebilir. [5]



impersonate ve kubectl auth can-i

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: alice-impersonator
rules:
- apiGroups: ["" ]
  resources: ["users"]
  verbs: ["impersonate"]
  resourceNames: ["alice"]
```

```
kubectl auth can-i get pods -n ekip-a
kubectl auth can-i get pods -n ekip-a --as=alice
```

- `resourceNames` , hedefi `alice` ile sınırlar; `users` kalıcı bir `User` API nesnesi değildir^[7]
- `kubectl auth can-i` , `SelfSubjectAccessReview` gönderir; `--as` varsa gerçek kullanıcı doğrulandıktan ve `impersonate` izni denetlendikten sonra hedef kimlik değerlendirilir^[3,7]
- Kullanıcı/grup impersonation izni cluster kapsamlıdır; ServiceAccount izni namespace kapsamlı verilebilir, fakat hedef hesabın diğer hakları korunur^[7]
- `yes` , yalnız yetkilendirme sonucudur; kabul denetimi veya nesne doğrulaması isteği yine reddedebilir^[1,17]

Token ve Sertifika Akışları





Pod'a bağlanan ServiceAccount token'i

- Kubernetes v1.22 ve sonrasında Pod'a sağlanan varsayılan kimlik bilgisi, TokenRequest ile alınan kısa ömürlü ve otomatik dönen token'dır; projected volume ile bağlanır ^[8,9]
- Pod, `serviceAccountName` belirtmezse namespace'in `default` ServiceAccount'ını kullanır; özel hesap görevleri ve etki alanını ayırır ^[8]
- RBAC etkinse `kube-system` dışındaki ServiceAccount'lar varsayılan API keşif izinleri dışında ayrıca yetki almaz ^[5]

```
spec:  
  serviceAccountName: raporlayici  
  automountServiceAccountToken: false
```

`automountServiceAccountToken: false` sadece API kimlik bilgisini otomatik olarak Pod'a bağlamaz; zaten varolan ServiceAccount kimliğini veya ona verilmiş RBAC kurallarını silmez. Alan hem ServiceAccount'ta hem Pod'da yazılırsa Pod değeri önceliklidir. ^[8,9]



CSR akışı

1. İstemci özel anahtarı yerel üretir; PKCS#10 isteğini `CertificateSigningRequest.spec.request` alanına koyar
2. `certificates.k8s.io/v1` nesnesi `signerName`, `usages` ve isteğe bağlı ömür talebiyle oluşturulur
3. Yetkili onaylayıcı subject, gruplar, signer, usages ve talep sahibini denetleyip onaylar ya da reddeder
4. Onay tek başına sertifika üretmez; uygun signer ayrıca imzalayıp `status.certificate` alanını doldurur
5. İstemci sertifikayı kubeconfig'te kullanır; kimlik doğrulandıktan sonra RBAC hâlâ her isteği yetkilendirir ^[2,10]

`kubernetes.io/kube-apiserver-client` signer'ı için kube-controller-manager otomatik onay vermez. İstemci CSR'ı oluşturma ve `certificatesigningrequests/approval` güncelleme izinlerinin birleşimi, keyfi kullanıcı adlarıyla sertifika oluşturabilme riski. ^[6,10]

Yetkilendirme Modları ve Namespace





ABAC ve Node Authorizer

Model	Politika kaynağı	Uygun mental model
ABAC	API server başlatılırken verilen satır bazlı JSON politika dosyası	Kullanıcı, grup, resource, namespace gibi nitelik eşleşirse izin verilir; değişiklik için politika dosyası ve API server yeniden başlatması gerekir
RBAC	Kubernetes API'deki rol ve binding nesneleri	Rol tabanlı, dinamik ve gözden geçirilebilir yetki yönetimi
Node	API server'ın özel yetkilendiricisi	<code>system:node:<nodeName></code> kubelet'ini kendi node'u ve o node'a bağlı Pod kaynaklarıyla sınırlar

Node authorizer kubelet'e gereken Pod, Secret, ConfigMap ve volume bilgilerini ilişkiye göre verir; `NodeRestriction` kabul eklentisi node ve Pod yazmalarını kendi kapsamına daraltan tamamlayıcı kontroldür. ^[11]

ABAC v1.36'da belgelenen bir yetkilendirme modu olsa da API nesnesiyle yönetilmez; yeni en az ayrıcalık tasarımı burada RBAC kullanılır. ^[3,12]



Namespace ve sınırları

- Namespace, namespaced nesneler için ad ve politika kapsamı sağlar; namespace'ler iç içe geçmez^[13]
- Node, PersistentVolume, StorageClass ve Namespace gibi cluster-scoped nesneler bu kapsamın dışındadır^[13]
- `kube-system` sistem nesnelerini barındırır; `kube-public` konvansiyonel olarak tüm istemcilerce okunabilir olacak biçimde tanımlanmıştır^[13]
- Aynı namespace'te workload oluşturabilen biri başka workload'ların Secret, ConfigMap, volume ve ServiceAccount kimlik bilgilerine dolaylı erişebilir; namespace içi sınırlar bu nedenle zayıf kabul edilir^[6]
- Namespace oluşturmak Pod trafiğini kendiliğinden izole etmez; NetworkPolicy ancak destekleyen ağ eklentisi ve seçici kurallarla etkili olur^[16]

Kaynaklar





Kaynaklar

1. Controlling Access to the Kubernetes API: kubernetes.io/docs/concepts/security/controlling-access/
2. Authenticating: kubernetes.io/docs/reference/access-authn-authz/authentication/
3. Authorization: kubernetes.io/docs/reference/access-authn-authz/authorization/
4. Admission Controllers: kubernetes.io/docs/reference/access-authn-authz/admission-controllers/
5. Using RBAC Authorization: kubernetes.io/docs/reference/access-authn-authz/rbac/
6. Role Based Access Control Good Practices: kubernetes.io/docs/concepts/security/rbac-good-practices/
7. User Impersonation: kubernetes.io/docs/reference/access-authn-authz/user-impersonation/
8. Service Accounts: kubernetes.io/docs/concepts/security/service-accounts/
9. Configure Service Accounts for Pods: kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/



Kaynaklar (devam)

10. Certificates and Certificate Signing Requests: kubernetes.io/docs/reference/access-authn-authz/certificate-signing-requests/
11. Using Node Authorization: kubernetes.io/docs/reference/access-authn-authz/node/
12. Using ABAC Authorization: kubernetes.io/docs/reference/access-authn-authz/abac/
13. Namespaces: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
14. Administer a Cluster / Namespaces: kubernetes.io/docs/tasks/administer-cluster/namespaces/
15. Resource Quotas: kubernetes.io/docs/concepts/policy/resource-quotas/
16. Network Policies: kubernetes.io/docs/concepts/services-networking/network-policies/
17. `kubectl auth can-i`: kubernetes.io/docs/reference/kubectl/generated/kubectl_auth/kubectl_auth_can-i/